

Here's how I'd write this if I were the CTO handing it to an engineering team.

Job Intelligence Agent

Technical Design Document (V0.1 MVP)

Author: CTO

Version: 0.1

Status: Draft

Objective: Validate that an automated job discovery and intelligence pipeline can consistently surface high-quality opportunities earlier than manual job searching, while preserving personalized applications.

1. Executive Summary

The goal of this project is **not** to automate job applications.

Instead, the system automates everything **before** the application:

- discovering jobs
- filtering irrelevant listings
- scoring opportunities
- enriching listings with company intelligence
- identifying resume gaps
- organizing follow-ups

The applicant remains responsible for submitting every application manually.

This avoids generic applications while dramatically reducing the time spent searching.

2. Product Philosophy

The product is a **Job Intelligence Agent**, not a job board.

Traditional workflow:

Search



Open 100 tabs



Read descriptions



Close 95 tabs



Apply to 5

Desired workflow:

Wake up



Receive today's top opportunities



Review



Personalize



Apply

The system should save hours every week while improving application quality.

3. Success Criteria

The MVP is successful if, over a 7-day period, it can:

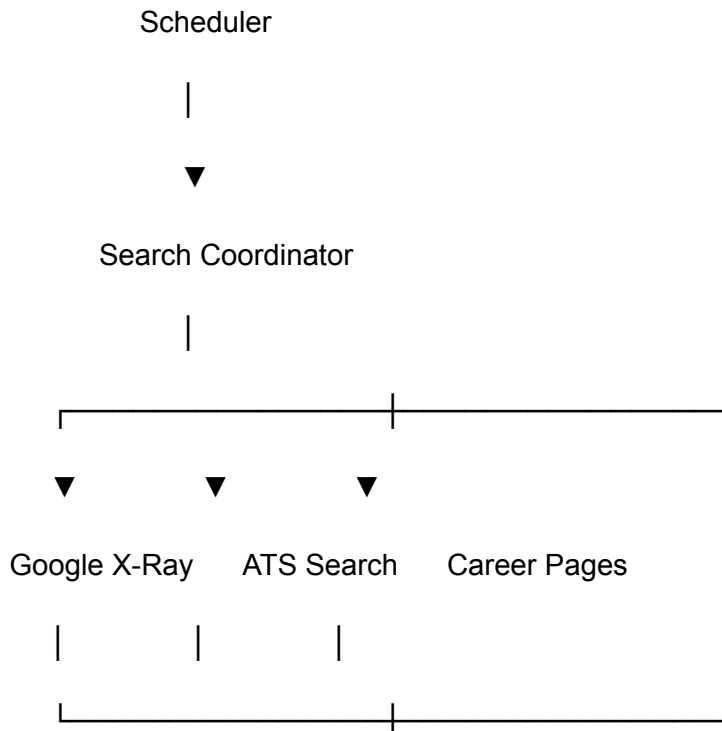
- discover new jobs automatically
- remove duplicates
- filter irrelevant roles
- rank jobs by relevance
- notify the user daily
- reduce manual search time by at least 80%

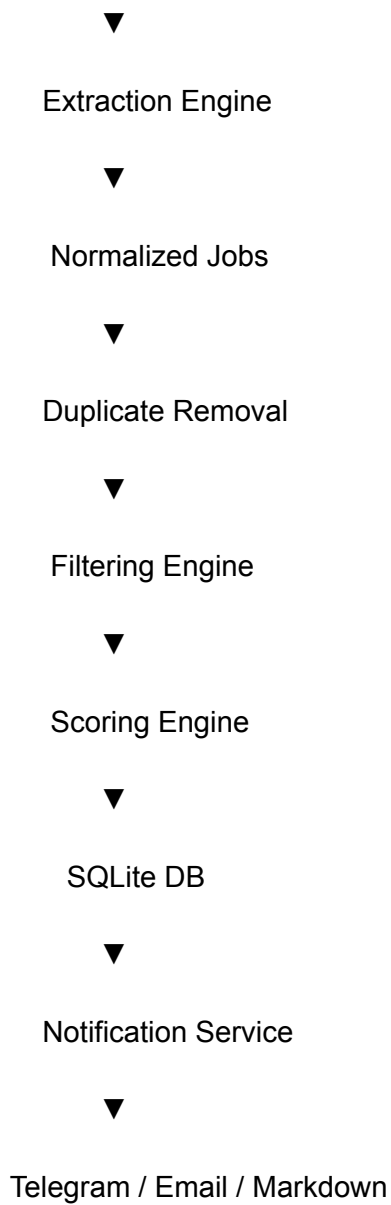
It is **not** required to:

- submit applications
- generate resumes
- write cover letters
- contact recruiters

Those belong in later phases.

4. High-Level Architecture





5. Technology Stack

Backend

NestJS

Reasons:

- modular architecture
 - dependency injection
 - scheduling support
 - queues
 - testing ecosystem
 - future scalability
-

Language

TypeScript

Strict mode enabled.

Database

SQLite

Reason:

- zero infrastructure
- fast
- portable
- sufficient for MVP

Migration to PostgreSQL should require minimal code changes.

Scheduler

NestJS Schedule

Runs every:

6 hours

Configurable.

HTTP

Axios

HTML Parsing

Cheerio

Primary parser.

Fallback:

Playwright

Only when required.

Notifications

Telegram Bot API

Future:

- Email
 - Slack
 - Discord
-

Logging

Pino

Validation

Zod

ORM

Drizzle ORM

Reasons:

- lightweight
 - TypeScript-first
 - SQLite support
 - PostgreSQL migration path
-

6. Folder Structure

src/

app/

search/

google/

greenhouse/

lever/

ashby/

wellfound/

parser/

extractor/

scoring/

filters/

notifier/

scheduler/

repositories/

entities/

config/

logger/

database/

scripts/

tests/

docs/

7. Modules

Search Module

Responsibility:

Generate search URLs.

Input:

roles

keywords

locations

sources

Output:

Search Tasks

Source Connectors

Each source implements the same interface.

```
interface JobSource {  
  
    search(query): Promise<JobListing[]>  
  
}
```

Supported:

- Google X-Ray
- Greenhouse
- Lever
- Ashby
- Wellfound

Future:

- YC
- Work at a Startup
- RemoteOK
- Company career pages

Extraction Module

Converts HTML into a normalized format.

Job

{

title

company

location

salary
description
url
postedDate
source
}

Deduplication Module

Hash:

company

+

title

+

location

Ignore duplicates.

Filtering Module

Reject jobs based on:

Experience

10+ years

Staff

Principal

Director

Location

Onsite

Visa required

Technology

C++

COBOL

SAP

etc.

Rules stored in configuration.

Scoring Module

Weighted score.

Example

React Native +30

React +15

TypeScript +15

Node +15

NestJS +15

Remote +20

Salary +10

Recently Posted +10

Returns

Score

Reasons

Matched Keywords

Missing Keywords

Storage Module

SQLite

Tables

Jobs

id

company

title

url

description

score

status

createdAt

updatedAt

Runs

INSERT

UPDATE

UPSERT

DELETE

Notification Module

Daily report.

Markdown.

Example

Top Jobs Today

1.

Acme

95%

Remote

Series A

Posted Today

2.

...

...

Send via Telegram.

8. Configuration

YAML

roles:

- React Native
- Frontend Engineer
- Software Engineer

keywords:

- TypeScript
- React
- Node
- NestJS

exclude:

- Senior Staff
- Principal
- COBOL

locations:

- Remote
- Worldwide

minimumScore:

80

No code changes required.

9. Scheduler

00:00

06:00

12:00

18:00

Pipeline

Search

↓

Extract

↓

Normalize

↓

Deduplicate

↓

Filter

↓

Score

↓

Store

↓

Notify

10. Logging

Every stage logs:

Started

Completed

Duration

Failures

Retries

11. Error Handling

Failures in one connector must not stop the pipeline.

Google failed

↓

Continue

Lever

↓

Continue

Ashby

↓

Continue

12. Phase 2 (AI)

Only after MVP validation.

New module

Resume Analyzer

Input

Resume

+

Job Description

Output

Match %

Missing Keywords

Resume Suggestions

No rewriting.

Only recommendations.

13. Phase 3

Company intelligence.

Gather

- funding
- founders
- hiring growth
- employee count
- tech stack

- latest news

Produces

Company Brief

14. Phase 4

Outreach assistant.

Generate

- recruiter email
- LinkedIn message
- follow-up message

Using company research.

15. Phase 5

Dashboard

Metrics

Jobs Found

Jobs Applied

Interview Rate

Response Rate

Average Match Score

Pending Follow-ups

16. Non-Functional Requirements

Performance

- Complete one run in under 5 minutes for 500 jobs.

Reliability

- One connector failure must not fail the pipeline.

Maintainability

- Every connector isolated.
- Configuration-driven rules.
- Unit tests for scoring and filtering.

Extensibility

- New job sources should implement a single interface.
 - Scoring weights configurable without code changes.
-

17. Future Vision

The long-term goal is **not** to become another job board.

The vision is to build an **AI-powered career operating system** that continuously works on behalf of the user:

- Monitoring thousands of companies and ATS platforms
- Identifying relevant opportunities within minutes of posting
- Ranking roles based on the user's evolving skills and preferences
- Researching companies and hiring teams
- Suggesting targeted resume improvements without fabricating experience
- Drafting personalized outreach and follow-up messages
- Tracking applications, interviews, and outcomes
- Learning from successes and rejections to improve future recommendations

Unlike traditional job boards that simply list vacancies, this system acts as an intelligence layer, helping professionals spend less time searching and more time applying strategically. The MVP described in this document is intentionally narrow: its purpose is to validate that automated discovery and prioritization create measurable value. Once that hypothesis is proven, more

advanced AI capabilities can be layered on incrementally without changing the core architecture.